DATABASE MANAGEMENT SYSTEMS

3rd Semester Final Exam • Complete Question Bank Answers

Modules 1–6 | ER Model | Relational Algebra | SQL | Normalization | PL/SQL | Transactions

Prepared from the uploaded DBMS question bank. Answers are written for exam revision, with examples and worked syntax.

How to use this guide

- Write theory answers in the order: definition -> explanation/key points -> example -> conclusion where useful.
- For SQL and PL/SQL questions, reproduce the syntax carefully and be ready to explain each clause.
- For ER diagrams, explicitly state cardinality (1:1, 1:N, M:N) and participation (total/partial).
- For normalization questions, state the functional dependencies and candidate key before decomposing. The STUDENT normalization answer makes its assumptions explicit because the question bank does not provide dependencies.
- SQL examples primarily use Oracle syntax because the question bank also covers PL/SQL. Some syntax differs in MySQL, PostgreSQL, and SQL Server.
- The “Electronics students” question does not provide a department field in the supplied Student/Course schemas, so the guide identifies the missing information rather than inventing it.

Contents

- MODULE 1: DBMS FUNDAMENTALS AND ER MODEL (16 questions)
- MODULE 2: RELATIONAL MODEL AND RELATIONAL ALGEBRA (12 questions)
- MODULE 3: SQL (16 questions)
- MODULE 4: RELATIONAL DATABASE DESIGN AND NORMALIZATION (9 questions)
- MODULE 5: PL/SQL (19 questions)
- MODULE 6: TRANSACTION MANAGEMENT AND QUERY PROCESSING (16 questions)

MODULE 1: DBMS FUNDAMENTALS AND ER MODEL

Core concepts, architecture, ER/EER design, keys, and database types.

Q1. Differentiate between the File System and Database Management System

A file system stores data in separate files controlled by application programs. A DBMS is software that defines, stores, retrieves, updates, and controls access to a database.

Key differences:

- **Redundancy:** file systems often duplicate the same facts across files; a DBMS can centralize data and reduce duplication.
- **Consistency:** file-based consistency must be enforced by each program; a DBMS uses constraints and transactions.
- **Data sharing:** file systems make multi-user sharing harder; a DBMS supports controlled concurrent access.
- **Security:** file permissions are usually coarse-grained; a DBMS can grant privileges on tables, views, or operations.
- **Data independence:** file formats are closely tied to programs; a DBMS separates logical data definitions from application code.
- **Backup and recovery:** often manual or application-specific in a file system; DBMSs provide recovery and backup facilities.
- **Querying:** file applications usually need custom code; a DBMS provides query languages such as SQL.
- **Cost and complexity:** file systems can be simple for small tasks; DBMSs have greater setup and administration overhead.

Example: storing student details in multiple spreadsheets can cause inconsistent addresses. A student database keeps one authoritative record and lets authorized applications use it.

Q2. Difference between Primary Key and Foreign Key

A primary key uniquely identifies each row in its own table. A foreign key is a column or set of columns that references a candidate/primary key in another table (or sometimes the same table).

- Primary key values must be unique and NOT NULL. A table normally has one primary-key constraint, which may contain multiple columns.
- Foreign-key values need not be unique; repeated values are common. They may be NULL unless NOT NULL is also specified.
- A primary key enforces entity integrity. A foreign key enforces referential integrity.

Example:

```
CREATE TABLE Department (
 Dept_ID NUMBER PRIMARY KEY,
 Dept_Name VARCHAR2(40) NOT NULL
);
CREATE TABLE Student (
 Student_ID NUMBER PRIMARY KEY,
 Student_Name VARCHAR2(60),
 Dept_ID NUMBER,
 CONSTRAINT fk_student_dept
  FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

Student.Student_ID is a primary key; Student.Dept_ID is a foreign key.

Q3. Describe Primary Key, Candidate Key, Super Key, Foreign Key, and Unique Key with examples

Use STUDENT(Student_ID, Email, Name, Dept_ID) as an example.

- **Super key:** any attribute set that uniquely identifies a row. {Student_ID} and {Student_ID, Name} are both super keys. A super key may include unnecessary attributes.
- **Candidate key:** a minimal super key. If Student_ID and Email are each unique, both are candidate keys.
- **Primary key:** the candidate key selected as the main identifier. Here, Student_ID could be selected. It must be unique and non-null.
- **Alternate key:** a candidate key not chosen as the primary key. If Student_ID is primary, Email is an alternate key.
- **Foreign key:** an attribute referencing a key in another table. Dept_ID may reference Department(Dept_ID).
- **Unique key/constraint:** requires the constrained value or combination to be unique. Example: Email VARCHAR2(100) UNIQUE. NULL handling for UNIQUE constraints depends on the DBMS; in Oracle, a unique constraint permits NULLs, while a primary key does not.

SQL example:

```
CREATE TABLE Student (
 Student_ID NUMBER PRIMARY KEY,
 Email VARCHAR2(100) UNIQUE,
 Name VARCHAR2(60) NOT NULL,
 Dept_ID NUMBER REFERENCES Department(Dept_ID)
);
```

Q4. Explain data abstraction and its different levels

Data abstraction hides unnecessary implementation details and shows each user only the information needed. The three classic DBMS levels are:

- **Physical/internal level:** describes how data is stored, such as files, pages, indexes, and access paths.
- **Logical/conceptual level:** describes what data is stored, its attributes, relationships, and constraints. Example: Student(Student_ID, Name, Dept_ID).
- **View/external level:** presents a tailored view to a user or application. Example: a lecturer sees student names and marks but not billing details.

Diagram:

```
External level: View A | View B | View C
          \   |   /
      Conceptual/logical schema
            |
        Internal/physical level
            |
         Stored data
```

Benefits include simpler use, reduced complexity, security, and support for data independence.

Q5. Explain data independence with an example

Data independence is the ability to change a schema at one level without requiring changes at the next higher level.

- **Physical data independence:** changes to physical storage do not require changes to the logical schema or application programs. Example: adding an index or moving a table to a different tablespace without changing the SQL used by the application.
- **Logical data independence:** changes to the conceptual schema do not require changes to external views or application programs, provided the information those views use remains available. Example: adding a new optional attribute to Student without changing an existing view that does not expose it.

Logical independence is generally harder to achieve than physical independence because applications often depend on the logical structure.

Q6. Compare 1-tier, 2-tier, and 3-tier architecture

- **1-tier:** the user interface, application logic, and database are on one system. Example: a developer using a local database directly. It is simple but less suitable for large multi-user deployments.
- **2-tier:** client application communicates directly with the database server, commonly using JDBC/ODBC. The client contains much of the presentation and business logic. It is easy to build but can expose database connectivity to many clients and is harder to scale securely.
- **3-tier:** presentation/client -> application or business-logic server -> database server. The middle tier validates requests and controls database access. It improves security, maintenance, and scalability but adds deployment complexity.

Example for a college portal: browser (presentation) -> web/API server (business logic) -> DB server (persistent records).

Q7. Roles and responsibilities of a Database Administrator (DBA)

A DBA manages the database environment to keep data available, secure, consistent, and performant. Main responsibilities:

- Install, configure, upgrade, and patch the DBMS.
- Design and maintain schemas, tablespaces, indexes, views, and storage.
- Create users, roles, privileges, authentication, and auditing policies.
- Plan backups, test restores, and manage disaster recovery.
- Monitor performance, inspect query plans, tune indexes and SQL, and manage resources.
- Enforce constraints, data integrity, and organizational standards.
- Manage concurrency, transactions, availability, and capacity.
- Apply patches and monitor vulnerabilities.
- Document changes, establish maintenance windows, and support developers/users.

The DBA balances performance, cost, availability, security, and recoverability.

Q8. Design an ER model for a Hospital Management System and identify relationship constraints

Possible ER design (attributes shown in parentheses):

- PATIENT(Patient_ID, Name, DOB, Gender, Phone, Address)
- DOCTOR(Doctor_ID, Name, Specialization, Phone)
- DEPARTMENT(Dept_ID, Dept_Name)
- APPOINTMENT(Appointment_ID, DateTime, Status, Patient_ID, Doctor_ID)
- ROOM(Room_ID, Type, ChargePerDay)
- ADMISSION(Admission_ID, AdmitDate, DischargeDate, Patient_ID, Room_ID, AttendingDoctor_ID)
- PRESCRIPTION(Prescription_ID, Date, Patient_ID, Doctor_ID)
- MEDICINE(Medicine_ID, Name, UnitPrice)
- PRESCRIPTION_ITEM(Prescription_ID, Medicine_ID, Dose, Duration)

Relationships and constraints:

- **Patient–Appointment:** one patient can have 0..N appointments; each appointment belongs to exactly one patient. (1:N)
- **Doctor–Appointment:** one doctor can handle 0..N appointments; each appointment has one doctor. (1:N)
- **Department–Doctor:** one department has 0..N doctors; each doctor belongs to one department (assumption). (1:N)
- **Patient–Admission:** one patient may have 0..N admissions over time; each admission is for one patient. (1:N)
- **Room–Admission:** one room can be used by many admissions over time, but an admission uses one room. (1:N over time; availability constraint prevents overlapping occupancy if single-bed rooms).
- **Doctor–Prescription and Patient–Prescription:** each prescription is issued by one doctor for one patient; each doctor/patient may have many prescriptions.
- **Prescription–Medicine:** many-to-many, resolved using PRESCRIPTION_ITEM; the associative entity stores dose and duration.

Participation: Appointment has total participation in both Patient and Doctor relationships; a Patient may have partial participation because they may not have booked an appointment. Cardinality and mandatory/optional participation should be shown using the notation taught in class. Additional business constraints include appointment time conflicts, valid discharge dates, and room availability.

Q9. Differentiate between Generalization and Specialization with examples

Both are EER concepts that organize entity types using an IS-A relationship.

- **Specialization is top-down:** divide a general entity into specialized subtypes based on distinguishing features. Example: EMPLOYEE -> DOCTOR, NURSE, ADMIN_STAFF. Subtypes inherit common Employee attributes and add their own.
- **Generalization is bottom-up:** combine similar entity types into a common supertype. Example: CAR and TRUCK are generalized into VEHICLE with common attributes such as Vehicle_ID and Registration_No.

Specialization often asks, ‘What subgroups exist within this entity?’ Generalization asks, ‘What common entity represents these similar types?’ Constraints may be disjoint/overlapping and total/partial.

Q10. Explain aggregation and inheritance in the EER model with an example

Inheritance means a subtype inherits the attributes and relationships of its supertype. Example: DOCTOR is a subtype of EMPLOYEE, so Doctor inherits Emp_ID, Name, and Address, and adds Specialization.

Aggregation treats a relationship set and its participating entities as a higher-level unit so that it can participate in another relationship. Example: DOCTOR treats PATIENT through the relationship TREATMENT; INSURANCE_COMPANY may cover that treatment. Aggregation lets the model express a relationship involving the whole Doctor–Treatment–Patient association rather than just one entity.

Inheritance models an IS-A relationship; aggregation models a HAS-A/relationship-as-a-unit abstraction.

Q11. Differentiate between strong and weak entity

- **Strong entity:** has its own primary key and can be uniquely identified independently. Example: EMPLOYEE(Emp_ID, Name).
- **Weak entity:** does not have a complete key of its own and depends on an owner (strong) entity for identification. It has a partial key/discriminator. Example: DEPENDENT(Dependent_Name, DOB) depends on EMPLOYEE. A dependent can be identified by (Emp_ID, Dependent_Name).

A weak entity has total participation in its identifying relationship with the owner. In ER notation, a strong entity is usually a single rectangle, a weak entity a double rectangle, an identifying relationship a double diamond, and its partial key is often shown with a dashed underline.

Q12. Different attributes in an ER diagram with examples

- **Simple/atomic:** cannot be meaningfully subdivided for the model; e.g., Salary.
- **Composite:** contains components; e.g., Address -> Street, City, PIN.
- **Single-valued:** one value per entity; e.g., DateOfBirth.
- **Multivalued:** multiple values; e.g., Employee phone numbers (often drawn as a double oval).
- **Stored:** physically recorded; e.g., DateOfBirth.
- **Derived:** calculated from other attributes; e.g., Age derived from DateOfBirth (often shown with a dashed oval).
- **Key attribute:** identifies an entity uniquely; e.g., Student_ID (underlined).
- **Optional/null attribute:** may have no value for some entity; e.g., MiddleName.
- **Complex attribute:** a combination of composite and multivalued attributes; e.g., multiple Addresses, each containing Street, City, and PIN.

An ER diagram should choose attributes according to the requirements and distinguish stored data from values that can reliably be derived.

Q13. Explain DBMS system architecture in detail

DBMS architecture can refer to both the three-schema architecture and the internal functional components.

Three-schema architecture:

- **External schema:** individual user/application views. Different users may see different subsets of the database.
- **Conceptual schema:** the complete logical structure, including entities, relationships, data types, and constraints.
- **Internal schema:** physical representation, file organization, indexes, and storage details.

Mappings between levels support data independence.

Major functional components:

- **Query processor:** parses and validates SQL, translates it into an internal form, optimizes it, and executes the chosen plan.
- **Storage manager:** manages data files, indexes, buffers, and disk access.
- **Transaction/concurrency manager:** coordinates concurrent transactions and enforces isolation.
- **Recovery manager:** uses logs/checkpoints or other recovery methods to restore consistency after failure.
- **Authorization/integrity manager:** checks privileges and enforces constraints.
- **System catalog/data dictionary:** stores metadata about tables, columns, users, constraints, and indexes.

Request flow: user SQL -> parser/validator -> optimizer -> execution engine -> storage/buffer manager -> stored database; security, transaction, and recovery services support the process.

Q14. Compare 1-tier, 2-tier, and 3-tier architecture in detail

1-tier: all parts run on the same machine; minimal network overhead and simple deployment, but weak separation of concerns and limited scaling. Typical example: a local desktop database.

2-tier: client application <-> database server. The client communicates directly using a database driver. Benefits: straightforward and responsive for small systems. Drawbacks: database credentials/connectivity may be distributed to clients, business rules may be duplicated, and many client connections can stress the database.

3-tier: client/presentation <-> application server/business logic <-> database server. Benefits: centralized business rules, stronger access control, easier updates, load balancing, and improved scalability. Drawbacks: more components to operate and debug; the application tier can become a bottleneck if poorly designed.

Use 1-tier for local/single-user work, 2-tier for controlled small client-server applications, and 3-tier for most larger web and enterprise systems.

Q15. What are cardinality constraints? Explain each

Cardinality specifies how many entities of one set can be associated with entities of another set.

- **One-to-one (1:**1): each A relates to at most one B and each B to at most one A. Example: Person–Passport, assuming each passport belongs to one person.
- **One-to-many (1:**N): one A may relate to many B, while each B relates to at most one A. Example: Department–Employee.
- **Many-to-one (N:**1): the same relationship viewed from the many side; many employees work in one department.
- **Many-to-many (M:**N): many A entities can relate to many B entities. Example: Student–Course. In a relational database this is commonly represented by a junction table Enrollment(Student_ID, Course_ID, ...).

Participation constraints add whether participation is total/mandatory or partial/optional. For example, each Enrollment must reference a student and course, but a student may have zero enrollments.

Q16. State different types of databases

Common classifications include:

- **Relational:** data in tables with keys and SQL; e.g., Oracle, PostgreSQL.
- **Hierarchical:** records organized as a tree with parent-child relationships.
- **Network:** records connected through more general graph-like links.
- **Object-oriented:** stores objects with identity, attributes, and behavior.
- **Object-relational:** relational tables extended with object-oriented features.
- **NoSQL:** a family including key-value, document, wide-column, and graph databases.
- **Centralized:** stored at one central site.
- **Distributed:** data resides across multiple networked sites.
- **Cloud database:** managed or hosted on cloud infrastructure.
- **Operational/transactional (OLTP):** supports day-to-day inserts/updates.
- **Analytical/data warehouse (OLAP):** optimized for reporting and historical analysis.

These categories overlap: a cloud-hosted relational database can also be distributed or used for OLTP.

MODULE 2: RELATIONAL MODEL AND RELATIONAL ALGEBRA

Definitions, mappings, algebraic operations, joins, and worked queries.

Q1. Define the relational model with an example

The relational model represents data as relations (tables). Each relation has tuples (rows) and attributes (columns). Values in a column come from a domain, and keys and integrity constraints maintain correctness. Relational operations produce relations as results.

Example: STUDENT(Student_ID, Name, Dept_ID)

```
Student_ID | Name | Dept_ID
101        | Asha | 10
102        | Ravi | 20
```

Student_ID can be the primary key; Dept_ID can reference DEPARTMENT(Dept_ID). The model is based on set-oriented data representation, although SQL tables can contain duplicate rows unless prevented by constraints or DISTINCT.

Q2. What is a relation? Explain its components

A relation is a table-like set of tuples following a relation schema. Its components are:

- **Relation name:** e.g., STUDENT.
- **Attributes:** named columns, e.g., Student_ID, Name.
- **Domain:** allowed values for an attribute, e.g., Student_ID is an integer domain.
- **Tuple:** one row representing an entity/record.
- **Degree:** number of attributes. STUDENT(Student_ID, Name, Dept_ID) has degree 3.
- **Cardinality:** number of tuples (rows) in the relation at a particular time.
- **Key:** attribute(s) that uniquely identify tuples.
- **NULL:** represents missing, unknown, or not-applicable information; it is not the same as zero or an empty string.

In the theoretical relational model, tuples are unique and their ordering is irrelevant; SQL implementations may allow duplicates and have no guaranteed row order without ORDER BY.

Q3. What is a relational database schema? Give an example

A relational database schema is the logical definition of a database: relation names, attributes, data types/domains, keys, and constraints. It differs from an instance, which is the actual data at a particular time.

Example schema:

- DEPARTMENT(Dept_ID PK, Dept_Name, HOD)
- STUDENT(Student_ID PK, Student_Name, Dept_ID FK -> DEPARTMENT.Dept_ID, Age)
- COURSE(Course_ID PK, Course_Name, Dept_ID FK -> DEPARTMENT.Dept_ID)
- ENROLL(Student_ID FK, Course_ID FK, Marks, PK(Student_ID, Course_ID))

The schema describes structure and rules; the rows currently stored form the database instance.

Q4. Explain the steps involved in mapping an ER model to a relational model

1. Strong entity: create one table with all simple attributes. Replace composite attributes with their components; choose the entity key as the primary key.

2. Weak entity: create a table with its attributes plus the owner entity's primary key as a foreign key. Use the owner key plus the weak entity's partial key as the primary key.
3. 1:1 relationship: place one side's primary key as a foreign key in the other side, preferably on the side with total participation; include relationship attributes. Alternatively use a separate table.
4. 1:N relationship: place the primary key of the '1' side as a foreign key in the 'N' side; include relationship attributes on the N side.
5. M:N relationship: create a separate relationship table containing both primary keys as foreign keys. Their combination is commonly the primary key; add relationship attributes.
6. Multivalued attribute: create a separate table with the entity key and multivalued attribute; use both as a key where appropriate.
7. N-ary relationship: create a separate table with foreign keys to participating entities and relationship attributes; choose a key based on cardinality constraints.
8. Specialization/generalization: use a superclass table plus subtype tables keyed by the superclass key, or another inheritance mapping strategy appropriate to disjoint/overlapping and total/partial constraints.
9. Add PK, FK, UNIQUE, NOT NULL, and CHECK constraints to preserve the ER model's rules.

Q5. Explain SELECT, PROJECT, UNION, INTERSECTION, and SET DIFFERENCE with examples

Let STUDENT(Student_ID, Name, Dept_ID) and assume relational algebra set semantics.

- **Selection σ chooses rows satisfying a predicate:** σ Dept_ID=10 (STUDENT) returns students in department 10.
- **Projection π chooses columns:** π Name,Dept_ID (STUDENT) returns only those attributes; duplicate tuples are removed under pure relational algebra.
- Union ∪ returns tuples present in either relation. The two relations must be union-compatible: same number of attributes and compatible domains. Example: π Student_ID (SPORTS_STUDENT) ∪ π Student_ID (MUSIC_STUDENT).
- Intersection ∩ returns tuples common to both compatible relations. Example: π Student_ID (SPORTS_STUDENT) ∩ π Student_ID (MUSIC_STUDENT).
- Set difference − returns tuples in the left relation but not the right. Example: π Student_ID (SPORTS_STUDENT) − π Student_ID (MUSIC_STUDENT).

These are fundamental relational algebra operations. SQL equivalents are SELECT with WHERE, column lists, UNION, INTERSECT, and MINUS/EXCEPT (the last name depends on the DBMS).

Q6. Describe the RENAME operation with an example

Rename (ρ) changes the name of a relation or its attributes, often to avoid ambiguity in self-joins or to make expressions clearer.

Examples:

- **Rename relation:** ρ S (STUDENT) gives the relation alias S.
- **Rename attributes:** ρ S(SID, SName, DID) (STUDENT) renames the relation to S and its attributes to SID, SName, DID.

In SQL, the related idea is a table alias, e.g. SELECT s.Name FROM Student s;. RENAME is a unary relational algebra operation.

Q7. Explain join operations, including inner and outer joins, with examples

A join combines tuples from two relations based on a condition. Let EMPLOYEE(Emp_ID, Name, Dept_ID) and DEPARTMENT(Dept_ID, Dept_Name).

- **Theta join:** joins using a general condition such as E.Salary > G.MinSalary.

- **Equi-join:** join condition uses equality, e.g. E.Dept_ID = D.Dept_ID; both join columns may appear in the result.
- **Natural join:** automatically matches columns with the same names and returns one copy of each matching join attribute; use carefully because unrelated same-named columns can cause accidental matches.
- **Inner join:** returns only matching pairs.

```
SELECT e.Name, d.Dept_Name
FROM Employee e INNER JOIN Department d
 ON e.Dept_ID = d.Dept_ID;
```

- **Left outer join:** returns every left-side employee, including employees without a department match; missing department columns are NULL.
- **Right outer join:** returns all departments, including departments with no employees.
- **Full outer join:** returns all matches plus unmatched rows from both sides.
- **Self join:** joins a table to itself, such as Employee to Manager using manager and employee IDs.

Outer joins preserve unmatched rows; inner joins do not.

Q8. Explain different types of constraints

Constraints are rules that restrict data to maintain integrity.

- **Domain constraint:** restricts data type/range, e.g. Age NUMBER CHECK (Age >= 0).
- **NOT NULL:** requires a value.
- **UNIQUE:** disallows duplicate non-NULL values according to DBMS semantics.
- **PRIMARY KEY:** uniquely identifies each row and does not allow NULL.
- **FOREIGN KEY:** requires a referenced parent key to exist, or permits NULL unless prohibited.
- **CHECK:** requires a Boolean condition to be true or unknown under SQL's NULL logic; examples include Salary > 0.
- **Entity integrity:** primary-key values cannot be NULL.
- **Referential integrity:** a foreign key must refer to an existing parent key, or be NULL if allowed.

Example:

```
CREATE TABLE Enroll (
 Student_ID NUMBER NOT NULL,
 Course_ID NUMBER NOT NULL,
 Marks NUMBER CHECK (Marks BETWEEN 0 AND 100),
 PRIMARY KEY (Student_ID, Course_ID),
 FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)
);
```

Q9. Explain different set operations

Relational algebra set operations are UNION (∪), INTERSECTION (∩), and SET DIFFERENCE (−). The operands must be union-compatible: same degree and corresponding attribute domains must be compatible.

- **UNION:** all tuples found in either operand.
- **INTERSECTION:** only tuples found in both operands.
- **DIFFERENCE:** tuples in the first operand but not the second; order matters, so A − B differs from B − A.

In SQL, UNION removes duplicates, while UNION ALL preserves them; INTERSECT performs intersection; Oracle uses MINUS for difference, while several other systems use EXCEPT. Set operators generally require matching column counts and compatible data types.

Q10. Describe mapping of ER and EER models to the relational model

ER mapping uses tables for entities and relationship tables/foreign keys for relationships. Strong entities become tables; weak entities include the owner's key; 1:N relationships use a foreign key on the N side; M:N relationships become junction tables; multivalued attributes become separate tables.

EER additional cases:

- **Superclass/subclass:** create a superclass table and a subclass table whose primary key is also a foreign key to the superclass. This preserves common attributes in one place.
- **Single-table mapping:** one table stores all superclass/subclass attributes plus a type discriminator; unused subtype attributes may be NULL.
- **Concrete-class mapping:** create tables for each concrete subtype, repeating inherited attributes; this may create redundancy.
- **Disjoint constraint:** an entity belongs to at most one subtype. Overlapping allows several subtypes. Total specialization means every superclass row belongs to a subtype; partial means some may not. The chosen mapping must preserve these constraints, sometimes with additional checks or triggers.
- **Aggregation:** map the aggregated relationship as a relation when it has its own attributes or must be referenced by another relationship.

Q11. Explain unary and binary relational operations

Unary operations act on one relation; binary operations combine two relations.

Unary examples:

- **Selection σ:** filters rows, σ Age>20(STUDENT).
- **Projection π:** selects columns, π Name,Age(STUDENT).
- **Rename ρ:** changes relation/attribute names, ρ S(STUDENT).

Binary examples:

- **Union R ∪ S:** tuples in either compatible relation.
- **Intersection R ∩ S:** tuples common to both.
- **Difference R − S:** tuples in R but not S.
- **Cartesian product R × S:** every row of R paired with every row of S.
- **Join R ⋈condition S:** combines related tuples and is commonly expressible through product plus selection.

Unary operations take one input; binary operations take two inputs. Projection and selection are often called fundamental unary operators; union, difference, product, and join are binary.

Q12. Relational algebra queries for the given Student, Department, Course, and Enroll schema

Given Student(Student_ID, Student_Name, Dept_ID, Age), Department(Dept_ID, Dept_Name, HOD), Course(Course_ID, Course_Name, Dept_ID), and Enroll(Student_ID, Course_ID, Marks).

1. Names of students in Computer Science:

```
π Student_Name ( Student ⋈ Student.Dept_ID=Department.Dept_ID
        σ Dept_Name='Computer Science'(Department) )
```

2. Names of students who scored more than 80 marks (in at least one enrollment):

```
π Student_Name ( Student ⋈ Student.Student_ID=Enroll.Student_ID
        σ Marks>80(Enroll) )
```

3. Names of courses offered by Electronics department:

```
π Course_Name ( Course ⋈ Course.Dept_ID=Department.Dept_ID
      σ Dept_Name='Electronics'(Department) )
```

4. Names of students enrolled in Database Management System:

```
π Student_Name ( Student ⋈ Student.Student_ID=Enroll.Student_ID
 (Enroll ⋈ Enroll.Course_ID=Course.Course_ID
  σ Course_Name='Database Management System'(Course)) )
```

5. Names of students older than 20:

```
π Student_Name ( σ Age>20(Student) )
```

If SQL-style natural-join notation is taught, the explicit equality conditions can be abbreviated, but showing join conditions makes the meaning unambiguous.

MODULE 3: SQL

SQL categories, functions, joins, constraints, transactions, views, and query-writing practice.

Q1. Explain DDL, DML, and DCL with examples

DDL (Data Definition Language) defines or changes database objects. Common commands: CREATE, ALTER, DROP, TRUNCATE, and sometimes RENAME. Example: `CREATE TABLE Student (Student_ID NUMBER PRIMARY KEY, Name VARCHAR2(60));`

DML (Data Manipulation Language) reads or changes data. Common commands: SELECT, INSERT, UPDATE, DELETE; some classifications call SELECT DQL. Example: `UPDATE Student SET Name='Asha' WHERE Student_ID=101;`

DCL (Data Control Language) manages access privileges. Common commands: GRANT and REVOKE. Example: `GRANT SELECT ON Student TO analyst_role;`

DDL handles structure, DML handles records, and DCL handles permissions. Exact categorization of SELECT and transaction behavior varies slightly across textbooks and DBMSs.

Q2. Short note on aggregate functions; explain any four

Aggregate functions calculate one value from a set of rows, often per group. NULL values are generally ignored by SUM, AVG, MIN, and MAX; COUNT(*) counts rows.

- **COUNT:** number of rows or non-NULL values. `SELECT COUNT(*) FROM Student;`
- **SUM:** total of numeric values. `SELECT SUM(Salary) FROM Employee;`
- **AVG:** arithmetic mean. `SELECT AVG(Marks) FROM Enroll;`
- **MIN:** smallest value. `SELECT MIN(Salary) FROM Employee;`
- **MAX:** largest value. `SELECT MAX(Marks) FROM Enroll;`

Example by department: `SELECT DNumber, AVG(Salary) FROM Employee GROUP BY DNumber;` returns one average per department.

Q3. What are the different joins in SQL? Explain with examples

Using Employee(EmpNo, Name, DNumber) and Department(DNumber, Dname):

- **INNER JOIN:** only matching rows. `SELECT e.Name,d.Dname FROM Employee e INNER JOIN Department d ON e.DNumber=d.DNumber;`
- **LEFT OUTER JOIN:** all employees and any matching department; departments absent for an employee appear as NULL.
- **RIGHT OUTER JOIN:** all departments and matching employees; departments with no employees still appear.
- **FULL OUTER JOIN:** all matched and unmatched rows from both sides.
- **CROSS JOIN:** Cartesian product; each employee is paired with every department.
- **SELF JOIN:** a table joins itself, often to display employee-manager pairs.
- **NATURAL JOIN:** joins automatically using same-named columns; use cautiously.
- **EQUI/THETA join:** classifications based on equality or general comparison conditions.

The most commonly used joins in applications are INNER JOIN and LEFT OUTER JOIN.

Q4. Problems based on SQL queries: core patterns to know

Use these patterns as templates and adapt table/column names to the question.

- **Filter rows:** `SELECT * FROM Student WHERE Age > 20;`
- **Sort results:** `SELECT Name, Salary FROM Employee ORDER BY Salary DESC;`

- **Count rows:** SELECT COUNT(*) FROM Employee;
- **Group and aggregate:** SELECT DNumber, AVG(Salary) FROM Employee GROUP BY DNumber;
- **Filter groups:** SELECT DNumber FROM Employee GROUP BY DNumber HAVING AVG(Salary) > 50000;
- **Join:** SELECT s.name,c.course_nm FROM Student s JOIN Course c ON c.courseId=s.courseId;
- **Subquery for maximum salary:** SELECT Name FROM Employee WHERE Salary=(SELECT MAX(Salary) FROM Employee);
- **Pattern matching:** SELECT Name FROM Student WHERE LOWER(Name) LIKE 'a%';
- **Insert:** INSERT INTO Student(Student_ID, Name) VALUES (101,'Asha');
- **Update:** UPDATE Employee SET Salary=Salary*1.05 WHERE EmpNo=10;
- **Delete:** DELETE FROM Student WHERE Student_ID=101;

Always check: correct join condition, filtering before aggregation versus HAVING after grouping, correct quotes for strings, and a WHERE clause on UPDATE/DELETE unless changing all rows is intended.

Q5. Explain SQL integrity constraints with examples

- **NOT NULL:** prevents a column being NULL. Name VARCHAR2(50) NOT NULL.
- **UNIQUE:** prevents duplicate values. Email VARCHAR2(100) UNIQUE.
- **PRIMARY KEY:** unique, non-NULL identifier. Student_ID NUMBER PRIMARY KEY.
- **FOREIGN KEY:** requires a matching referenced key. Dept_ID REFERENCES Department(Dept_ID).
- **CHECK:** requires an expression such as CHECK (Marks BETWEEN 0 AND 100).
- **DEFAULT:** supplies a value when one is not supplied, e.g. Status VARCHAR2(10) DEFAULT 'ACTIVE' (DEFAULT is a column default rather than a validation constraint in every classification).

Example:

```
CREATE TABLE Student (
 Student_ID NUMBER CONSTRAINT pk_student PRIMARY KEY,
 Name VARCHAR2(60) NOT NULL,
 Email VARCHAR2(100) UNIQUE,
 Age NUMBER CHECK (Age >= 0),
 Dept_ID NUMBER,
 CONSTRAINT fk_student_dept FOREIGN KEY (Dept_ID)
  REFERENCES Department(Dept_ID)
);
```

Constraints enforce rules in the database itself, rather than relying only on application code.

Q6. Explain TCL commands

TCL (Transaction Control Language) commands manage transaction boundaries.

- **COMMIT:** permanently makes the transaction's changes visible/durable according to DBMS guarantees.
- **ROLLBACK:** undoes uncommitted changes in the current transaction, or to a savepoint.
- **SAVEPOINT:** marks a point within a transaction.
- **ROLLBACK TO SAVEPOINT:** undoes changes after that point while preserving earlier changes.
- **SET TRANSACTION:** sets transaction properties, such as isolation/access mode where supported.

Example (transaction behavior and DDL auto-commit details are DBMS-dependent):

```
UPDATE Account SET Balance=Balance-500 WHERE Account_ID=1;
SAVEPOINT after_debit;
UPDATE Account SET Balance=Balance+500 WHERE Account_ID=2;
COMMIT;
```

If the second operation fails, the application can roll back appropriately. Transfers should be handled as one atomic transaction.

Q7. Short note on database triggers

A trigger is stored program logic that executes automatically when a specified event occurs. Triggers can run BEFORE or AFTER INSERT, UPDATE, or DELETE, and can be row-level (once per affected row) or statement-level (once per statement), depending on the DBMS.

Uses: enforcing complex business rules, auditing changes, maintaining derived information, and preventing invalid updates. Example: a BEFORE UPDATE trigger rejects a negative salary.

Advantages: centralizes enforcement and runs regardless of which application performs the change. Disadvantages: hidden side effects, harder debugging, possible performance costs, and accidental recursion or mutating-table problems if poorly written. Prefer declarative constraints for simple rules such as NOT NULL, UNIQUE, and CHECK.

Q8. Explain different string operations in SQL

Common string functions (exact names vary by DBMS):

- **UPPER(s):** uppercase; `UPPER(Name)`.
- **LOWER(s):** lowercase; `LOWER(Name)`.
- **INITCAP(s):** capitalize words in Oracle.
- **LENGTH(s):** number of characters; `LENGTH(Name)`.
- **SUBSTR(s,start,len):** extract part of a string in Oracle; `SUBSTR(Name,1,3)`.
- **CONCAT(a,b) or `a || b`:** join strings; Oracle supports `Name || ' - ' || Email`.
- **TRIM(s), LTRIM(s), RTRIM(s):** remove spaces/characters at the ends.
- **INSTR(s,sub):** position of substring in Oracle.
- **REPLACE(s,old,new):** replace matching text.
- **LIKE with `%` and `_`:** pattern matching, e.g. `Name LIKE 'A%'` starts with A; _ matches one character.

Example: `SELECT UPPER(Name), LENGTH(Name) FROM Student;`. Case sensitivity and function names can differ between platforms.

Q9. Explain ORDER BY and GROUP BY with examples

ORDER BY sorts the result rows; ascending is the default and DESC sorts descending. It is normally evaluated after filtering and grouping.

```
SELECT Name, Salary FROM Employee
ORDER BY Salary DESC, Name ASC;
```

GROUP BY combines rows sharing the same grouping values so aggregates can be calculated per group.

```
SELECT DNumber, COUNT(*) AS EmployeeCount, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY DNumber;
```

WHERE filters rows before grouping; HAVING filters groups after aggregates are calculated.

```
SELECT DNumber, AVG(Salary)
FROM Employee
WHERE Gender='Female'
GROUP BY DNumber
HAVING AVG(Salary) > 50000;
```

Columns in SELECT that are not aggregate expressions generally must appear in GROUP BY.

Q10. Describe GRANT and REVOKE with examples

GRANT gives permissions; REVOKE removes previously granted permissions. Privileges may apply to system actions or specific database objects depending on the DBMS.

Examples (Oracle-style):

```
GRANT SELECT, INSERT ON Student TO lab_user;
REVOKE INSERT ON Student FROM lab_user;
```

A DBA or object owner generally must have authority to grant the permission. Granting WITH GRANT OPTION (where supported) lets a recipient grant an object privilege to others. Follow least privilege: provide only the permissions required for a role.

Q11. Explain ALTER command with two examples

ALTER changes the structure of an existing database object. Examples:

```
-- 1. Add a column
ALTER TABLE Student ADD (City VARCHAR2(40));


-- 2. Change a column's allowed size/type (syntax varies)
ALTER TABLE Student MODIFY (Name VARCHAR2(100));
```

Other ALTER TABLE operations can add/drop constraints, rename columns, or drop columns. ALTER changes the schema, not the individual row values directly. DDL transaction behavior varies across DBMSs; Oracle issues implicit commits around DDL.

Q12. How do you create a view in SQL?

A view is a named query that presents selected data like a virtual table. It can simplify complex queries and restrict exposure of sensitive columns. A normal view stores its query definition, not a separate copy of its result rows (materialized views are different).

Example:

```
CREATE VIEW ActiveStudents AS
SELECT Student_ID, Name, Dept_ID
FROM Student
WHERE Status = 'ACTIVE';


SELECT * FROM ActiveStudents;
```

Views can be updateable only when the DBMS can map the modification unambiguously to underlying tables and no disqualifying features are present. A view is not automatically a security boundary unless permissions are configured appropriately.

Q13. Purpose of JOIN; compare INNER, LEFT, RIGHT, and FULL OUTER JOIN

A JOIN combines rows from related tables based on a condition. Let Employee(EmpNo, Name, DNumber) be the left table and Department(DNumber, Dname) the right table.

```
-- Only matching employees/departments
SELECT e.Name, d.Dname
FROM Employee e INNER JOIN Department d ON e.DNumber=d.DNumber;

-- Every employee, even without a matching department
SELECT e.Name, d.Dname
FROM Employee e LEFT OUTER JOIN Department d ON e.DNumber=d.DNumber;

-- Every department, even if it has no employees
SELECT e.Name, d.Dname
FROM Employee e RIGHT OUTER JOIN Department d ON e.DNumber=d.DNumber;

-- Every employee and every department; unmatched side is NULL
SELECT e.Name, d.Dname
FROM Employee e FULL OUTER JOIN Department d ON e.DNumber=d.DNumber;
```

INNER returns matches only. LEFT preserves all left rows. RIGHT preserves all right rows. FULL preserves rows from both sides. Oracle supports outer joins using ANSI JOIN syntax; older (+) syntax exists but is not preferred. Some systems do not directly support FULL OUTER JOIN.

Q14. Explain the role of DDL, DML, and DCL; commands and practical examples

DDL defines database structure.

- **CREATE:** create an object: CREATE TABLE Course (Course_ID NUMBER PRIMARY KEY, Course_Name VARCHAR2(80));
- **ALTER:** modify an object: ALTER TABLE Course ADD (Credits NUMBER);
- **DROP:** remove an object definition and its data: DROP TABLE Course;
- **TRUNCATE:** remove all rows efficiently while retaining the table structure: TRUNCATE TABLE Course; (behavior/rollback varies by DBMS).

DML manipulates table data.

- **INSERT:** INSERT INTO Course(Course_ID,Course_Name) VALUES (1,'DBMS');
- **SELECT (often classified separately as DQL):** SELECT Course_Name FROM Course;
- **UPDATE:** UPDATE Course SET Course_Name='Database Systems' WHERE Course_ID=1;
- **DELETE:** DELETE FROM Course WHERE Course_ID=1;

DCL controls permissions.

- **GRANT:** GRANT SELECT ON Course TO student_role;
- **REVOKE:** REVOKE SELECT ON Course FROM student_role;

Remember: DDL = object definitions, DML = records, DCL = permissions. TCL such as COMMIT and ROLLBACK manages transactions.

Q15. SQL queries for Employee and Department tables

Given EMPLOYEE(EmpNo, Name, DoB, Address, Gender, Salary, DNumber) and DEPARTMENT(DNumber, Dname, ManagerEmpNo, ManagerStartDate). The examples use Oracle SQL.

i) Display the age of male employees:

```
SELECT Name, TRUNC(MONTHS_BETWEEN(SYSDATE, DoB)/12) AS Age
FROM Employee
WHERE UPPER(Gender) = 'MALE';
```

ii) Display all employees in Marketing:

```
SELECT e.*
FROM Employee e JOIN Department d ON d.DNumber=e.DNumber
WHERE d.Dname='Marketing';
```

iii) Name of the female employee with the highest salary (returns all tied highest-paid females):

```
SELECT Name
FROM Employee
WHERE UPPER(Gender)='FEMALE'
 AND Salary=(SELECT MAX(Salary) FROM Employee
        WHERE UPPER(Gender)='FEMALE');
```

iv) Oldest manager in the company (manager here means an employee listed as a department manager):

```
SELECT e.Name
FROM Employee e
JOIN Department d ON d.ManagerEmpNo=e.EmpNo
ORDER BY e.DoB ASC
FETCH FIRST 1 ROW ONLY;
```

This returns one person if tied; use `RANK()` or a correlated subquery to return all ties.

v) Department name of employee SMITH:

```
SELECT d.Dname
FROM Employee e JOIN Department d ON d.DNumber=e.DNumber
WHERE UPPER(e.Name)='SMITH';
```

Assumptions: DoB is a DATE, Gender stores values such as Male/Female, and names may not be unique. In real systems, use EmpNo to identify an individual.

Q16. SQL statements for Student and Course tables

Given Student(Enrno, name, courseId, emailId, cellno) and Course(courseId, course_nm, duration).

vi) Add city to Student:

```
ALTER TABLE Student ADD (city VARCHAR2(50));
```

vii) List students enrolled in a Computer course:

```
SELECT s.*
FROM Student s JOIN Course c ON c.courseId=s.courseId
WHERE LOWER(c.course_nm) LIKE '%computer%';
```

If the intended condition is an exact course name, use `LOWER(c.course_nm)='computer'`.

viii) List all courses with duration:

```
SELECT course_nm, duration FROM Course;
```

ix) List students whose names start with A:

```
SELECT * FROM Student WHERE LOWER(name) LIKE 'a%';
```

x) Email and cell number of all Electronics students:

The provided schema does not include a department/branch attribute in Course, so this cannot be determined from only the two listed tables. If Course includes `department_name`, use:

```
SELECT s.emailId, s.cellno
FROM Student s JOIN Course c ON c.courseId=s.courseId
WHERE LOWER(c.department_name)='electronics';
```

MODULE 4: RELATIONAL DATABASE DESIGN AND NORMALIZATION

Normal forms, functional dependencies, closures, and decomposition.

Q1. What is normalization? Explain its need

Normalization is the systematic organization of relational tables and dependencies to reduce avoidable redundancy and prevent insertion, update, and deletion anomalies. It decomposes relations based on functional dependencies while aiming to preserve information and constraints.

Why it is needed:

- **Update anomaly:** the same fact appears in several rows and all copies must be changed.
- **Insertion anomaly:** a fact cannot be inserted without another unrelated fact.
- **Deletion anomaly:** deleting one row unintentionally deletes the only copy of another fact.
- It improves data consistency and makes dependencies clearer.

Normalization is not simply 'make more tables'; decomposition should ideally be lossless and dependency-preserving, and performance considerations may sometimes justify controlled denormalization.

Q2. Explain 1NF, 2NF, 3NF, and BCNF with examples

1NF (First Normal Form): each cell contains a single atomic value; there are no repeating groups or multivalued columns. Example: replace Phone1, Phone2 or a list of phone numbers in one cell with separate rows in StudentPhone(Student_ID, Phone).

2NF: relation is in 1NF and every non-prime attribute is fully functionally dependent on the whole of every candidate key; no partial dependency on only part of a composite candidate key. Example: ENROLL(Student_ID, Course_ID, Student_Name, Course_Name, Marks), key (Student_ID, Course_ID). Student_ID -> Student_Name and Course_ID -> Course_Name are partial dependencies. Decompose into STUDENT(Student_ID, Student_Name), COURSE(Course_ID, Course_Name), and ENROLL(Student_ID, Course_ID, Marks).

3NF: relation is in 2NF and has no problematic transitive dependency of non-prime attributes on keys. Formal test: for every nontrivial FD X -> A, X is a superkey or A is a prime attribute. Example: EMP(Emp_ID, Dept_ID, Dept_Name) with Emp_ID -> Dept_ID and Dept_ID -> Dept_Name. Decompose EMPLOYEE(Emp_ID, Dept_ID) and DEPARTMENT(Dept_ID, Dept_Name).

BCNF: for every nontrivial functional dependency X -> Y, X must be a superkey. BCNF is stricter than 3NF. Example: R(Student, Course, Instructor), with (Student, Course) -> Instructor and Instructor -> Course. Instructor is not a superkey, so BCNF is violated even though the relation can satisfy 3NF if Course is prime. Decompose into INSTRUCTOR_COURSE(Instructor, Course) and STUDENT_INSTRUCTOR(Student, Instructor), subject to the stated FDs.

Q3. Does 3NF allow redundancy? Justify

Yes. 3NF can allow some redundancy in cases where a functional dependency has a determinant that is not a superkey but the dependent attribute is prime (part of at least one candidate key). That is one reason 3NF is less restrictive than BCNF.

Example: R(Student, Course, Instructor) with FDs (Student, Course) -> Instructor and Instructor -> Course. Candidate keys are (Student, Course) and (Student, Instructor), so Course is a prime attribute. The FD Instructor -> Course violates BCNF because Instructor is not a superkey, but it can satisfy the 3NF test because Course is prime. Repeated Instructor/Course facts can create redundancy.

Thus, 3NF balances redundancy reduction with dependency preservation; BCNF removes more redundancy but may not preserve all dependencies after decomposition.

Q4. What is BCNF? Explain with a suitable example

A relation is in Boyce-Codd Normal Form (BCNF) if, for every nontrivial functional dependency X -> Y, X is a superkey. 'Nontrivial' means Y is not contained in X. BCNF eliminates certain anomalies left in 3NF.

Example: R(Student, Course, Instructor), with FDs (Student, Course) -> Instructor and Instructor -> Course. The candidate keys are (Student, Course) and (Student, Instructor). Instructor alone does not uniquely identify a row, so Instructor -> Course violates BCNF.

Decomposition:

- INSTRUCTOR_COURSE(Instructor, Course) where Instructor is the key.
- STUDENT_INSTRUCTOR(Student, Instructor).

This removes the offending dependency from the larger relation. In general, BCNF decomposition is lossless when performed using a violating FD, but it may not preserve every original functional dependency; verify the exact dependencies and join behavior for a particular example.

Q5. What is decomposition? What are its properties?

Decomposition splits a relation R into smaller relations R1, R2, ... to reduce redundancy and anomalies. A good decomposition should have:

- **Lossless join:** joining the decomposed relations reconstructs exactly the original relation, without losing tuples or creating spurious tuples.
- **Dependency preservation:** the original functional dependencies can be enforced by checking dependencies on individual decomposed relations, without needing to join them.
- **Attribute preservation:** all original attributes appear across the decomposed schemas.
- **Reduced redundancy/anomalies:** the purpose of normalization.

Losslessness is essential for preserving information. Dependency preservation is highly desirable for efficient constraint enforcement, but BCNF decomposition does not always guarantee it.

Q6. Compare lossy and lossless decomposition

Lossless decomposition: a natural join of the decomposed relations produces exactly the original relation. No information is lost and no spurious rows appear.

Lossy decomposition: joining the decomposed tables can produce extra spurious tuples or fail to reconstruct the exact original relation. This changes the information represented by the original table.

Example of lossless decomposition: R(A,B,C) with A -> B can be decomposed into R1(A,B) and R2(A,C). Their common attribute A functionally determines all attributes of R1 (A -> AB), so joining R1 and R2 on A reconstructs R.

A decomposition may be lossless but not dependency-preserving, or dependency-preserving but lossy. Evaluate the properties separately.

Q7. Conditions for lossless decomposition with example

For a binary decomposition of R into R1 and R2, a common test for losslessness under functional dependencies F is that the common attributes R1 ∩ R2 functionally determine all attributes of at least one component: (R1 ∩ R2) -> R1 or (R1 ∩ R2) -> R2 must be implied by F. Equivalently, the common attributes form a superkey of at least one component.

Example: R(A,B,C), F={A -> B}. Decompose into R1(A,B) and R2(A,C). The common attribute is A. Since A -> B, A -> AB, so A determines all of R1. Therefore the decomposition is lossless.

Contrast: if R(A,B,C) is decomposed into R1(A,B) and R2(B,C) without dependencies that let B determine a component, the natural join may combine unrelated A values with C values sharing B, producing spurious tuples. Test using the given functional dependencies rather than judging from the table names.

Q8. Normalize STUDENT(RollNo, Name, Course, Instructor, Dept) up to 3NF

Assume the following business rules (the question does not specify FDs, so state assumptions in the exam):

- RollNo -> Name
- Course -> Instructor
- Instructor -> Dept
- A student can take many courses and each course can have many students. Therefore candidate key is (RollNo, Course).

UNF/1NF: assume every field is atomic and each row represents one student-course enrollment. The relation is already in 1NF if Course is one value per row.

Original relation: STUDENT_COURSE(RollNo, Name, Course, Instructor, Dept). Key = (RollNo, Course).

2NF: remove partial dependencies on components of the composite key. RollNo -> Name and Course -> Instructor, Dept (via Course -> Instructor and Instructor -> Dept) are not dependent on the whole key. Decompose to:

- STUDENT(RollNo PK, Name)
- COURSE_2NF(Course PK, Instructor, Dept)
- ENROLLMENT(RollNo FK, Course FK, PK(RollNo, Course))

3NF: COURSE_2NF still has a transitive dependency Course -> Instructor -> Dept. Remove it:

- STUDENT(RollNo PK, Name)
- COURSE(Course PK, Instructor FK)
- INSTRUCTOR(Instructor PK, Dept)
- ENROLLMENT(RollNo FK, Course FK, PK(RollNo, Course))

Now each non-key fact is stored in the relation determined by its key. This result depends on the assumed FDs; if the course, instructor, or department rules differ, the decomposition may differ too.

Q9. For R(A,B,C), with A -> B and B -> C, find A+ and candidate keys

Attribute closure A+ is the set of attributes functionally determined by A. Start with A+ = {A}. From A -> B, add B: A+ = {A,B}. From B -> C, add C: A+ = {A,B,C}. No more attributes remain, so A+ = {A,B,C}.

Since A+ contains all attributes of R, A is a superkey. It is minimal because the only proper subset of {A} is the empty set, which does not determine all attributes. Therefore A is a candidate key.

B+ = {B,C}, and C+ = {C}; neither determines A. Thus, under only the given FDs, the only candidate key is A.

MODULE 5: PL/SQL

Procedural blocks, variables, stored programs, cursors, and triggers.

Q1. Explain the advantages of PL/SQL over SQL

PL/SQL is Oracle's procedural extension to SQL. Advantages over plain SQL include:

- Combines multiple SQL statements, variables, decisions, and loops into one block.
- Reduces client-server network traffic by sending a block/procedure to the database.
- Supports exception handling for runtime errors.
- Supports modular programming using procedures, functions, packages, and triggers.
- Enables conditional and iterative logic that plain SQL queries do not express directly.
- Encourages code reuse and centralized business logic.
- Can improve security by granting permission to execute a stored program without giving direct access to underlying tables.

SQL remains best for declarative data retrieval and modification; PL/SQL adds procedural control around SQL. Performance depends on design and workload, not merely on using PL/SQL.

Q2. Compare SQL and PL/SQL with respect to execution, features, and error handling

SQL is a declarative language: specify what data is required or changed. A SQL statement is parsed and executed by the database engine. It does not provide general-purpose loops or block-level exception handlers.

PL/SQL is a procedural/block-structured language that embeds SQL. A PL/SQL block can contain declarations, executable statements, control flow, cursors, and exception handlers. The PL/SQL engine processes procedural statements and submits SQL statements to the SQL engine.

Execution: SQL executes a statement; PL/SQL executes a block that may contain many SQL statements.

Features: SQL focuses on querying and data manipulation; PL/SQL adds variables, IF/CASE, loops, records, procedures, functions, packages, and triggers.

Error handling: SQL reports statement errors to the caller; PL/SQL supports `EXCEPTION ... WHEN ... THEN` handlers and can raise application errors.

Example: `SELECT salary INTO v_salary FROM employee WHERE empno=1;` inside PL/SQL stores a query result in a variable and exceptions can handle no row or multiple rows.

Q3. Explain the architecture of PL/SQL with a neat diagram

A PL/SQL block is submitted to the Oracle database. The PL/SQL engine processes procedural constructs, while SQL statements are sent to the SQL engine for parsing, optimization, and execution. Both interact with database services and stored objects.

```
Client / Application
      | PL/SQL block or call
      v
+-------------------------+
| PL/SQL Engine           |  declarations, IF, loops, exceptions
+-------------------------+
      | SQL statements
      v
+-------------------------+
| SQL Engine              |  parse, optimize, execute SQL
+-------------------------+
      |
      v
Database objects / buffer cache / data files
```

If a block contains many SQL statements, moving the block to the server can reduce network round trips. Stored procedures and functions are compiled and stored in the database; they can be invoked by applications or other PL/SQL code.

Q4. Explain character sets supported in PL/SQL with examples

PL/SQL source uses several classes of characters:

- **Letters:** A–Z and a–z are used in identifiers and keywords; identifiers may have additional rules depending on quoting.
- **Digits:** 0–9 are used in numeric literals and identifiers where permitted.
- **Whitespace:** spaces, tabs, and line breaks separate lexical units.
- **Special symbols:** operators and punctuation such as `+ - * / = ; , . ( ) :` and `%`.
- **Quotation marks:** single quotes delimit string literals; double quotes can delimit quoted identifiers.

Examples: `v_total NUMBER := 125;`, `v_name VARCHAR2(20) := 'Asha';`, and `-- comment`. National/multibyte text depends on the database character set and data type, such as CHAR/VARCHAR2 versus NCHAR/NVARCHAR2 in Oracle. Character repertoire and identifier rules should follow the database configuration/version.

Q5. Explain the lexical units of PL/SQL with examples

Lexical units are the smallest meaningful elements of source code.

- **Keywords/reserved words:** `DECLARE`, `BEGIN`, `IF`, `THEN`, `END`, `EXCEPTION`.
- **Identifiers:** user-defined names such as `v_salary`, `emp_count`, `raise_salary`.
- **Literals:** fixed values such as `100`, `3.14`, `'Hello'`, `TRUE`, or a date literal.
- **Delimiters and punctuation:** `;`, `,`, `(`, `)`, `:=`, `=>`, `..` A delimiter separates or groups tokens.
- **Operators:** arithmetic, comparison, logical, and assignment operators, e.g. `+`, `=`, `AND`, `:=`.
- **Comments:** `-- one-line comment` and `/* multi-line comment */`.

Example: in `v_count := 10; -- initialise`, `v_count` is an identifier, `:=` is assignment, `10` is a numeric literal, `;` is a delimiter, and the text after `--` is a comment.

Q6. Identify keywords, identifiers, literals, delimiters, and comments in a PL/SQL block

Consider:

```
DECLARE
 v_salary NUMBER := 50000; -- starting salary
BEGIN
 IF v_salary > 0 THEN
  DBMS_OUTPUT.PUT_LINE('Valid');
 END IF;
END;
/
```

- **Keywords:** DECLARE, BEGIN, IF, THEN, END.
- **Identifiers:** v_salary, NUMBER (a predefined type name), DBMS_OUTPUT, PUT_LINE.
- **Literals:** 50000, 0, and the string 'Valid'.
- **Delimiters:** ;, ., parentheses, and the slash / used by tools such as SQL*Plus to submit the block. := is an assignment operator.
- **Comment:** -- starting salary.

When answering a token-identification question, quote the specific token and label its category. A token can be a predefined type or built-in package name rather than a user-created identifier.

Q7. Explain scope and visibility of identifiers in nested PL/SQL blocks

Scope is the region of a program in which a declaration is valid. Visibility is whether a name can be referenced at a particular point. An identifier declared in an outer block is generally visible in its inner blocks unless hidden by a declaration with the same name. An inner declaration is not visible outside its block.

Example:

```
DECLARE
 v_message VARCHAR2(20) := 'outer';
BEGIN
 DECLARE
  v_count NUMBER := 1;
  v_message VARCHAR2(20) := 'inner'; -- hides outer name here
 BEGIN
  DBMS_OUTPUT.PUT_LINE(v_message); -- inner
  DBMS_OUTPUT.PUT_LINE(v_count);
 END;
 DBMS_OUTPUT.PUT_LINE(v_message); -- outer
 -- v_count is out of scope here
END;
/
```

Use meaningful names and avoid unnecessary shadowing. Block labels and qualified names can help reference an outer declaration when needed.

Q8. Explain different methods of assigning values to variables in PL/SQL

Common ways include:

- **Direct assignment in executable section using `:**=: v_total := 10;`.
- **Initialization/default at declaration:** v_rate NUMBER := 0.05; or v_rate NUMBER DEFAULT 0.05;.

- **SELECT INTO:** fetch a query result into variables: `SELECT salary INTO v_salary FROM employee WHERE empno=10;`. The query must return exactly one row; zero rows raises NO_DATA_FOUND and multiple rows raise TOO_MANY_ROWS.
- **Assignment from expressions:** `v_total := v_price * v_quantity;`.
- **Assignment through parameters:** IN parameters receive values; OUT/IN OUT parameters can pass computed values back to the caller.
- **Cursor fetch:** explicit cursor rows can be fetched into variables or records.

Always declare a variable with an appropriate data type; `%TYPE` and `%ROWTYPE` can match table columns/rows and reduce type mismatches.

Q9. Write a PL/SQL block using direct assignment and SELECT INTO

This Oracle PL/SQL example uses direct assignment for one variable and SELECT INTO for an employee salary. Replace employee number 100 with a valid row in the table.

```
SET SERVEROUTPUT ON;
DECLARE
 v_bonus_rate NUMBER := 0.10; -- direct initialization
 v_salary     Employee.Salary%TYPE;
 v_bonus      NUMBER;
BEGIN
 SELECT Salary INTO v_salary
 FROM Employee
 WHERE EmpNo = 100;

 v_bonus := v_salary * v_bonus_rate; -- expression assignment
 DBMS_OUTPUT.PUT_LINE('Salary = ' || v_salary);
 DBMS_OUTPUT.PUT_LINE('Bonus = ' || v_bonus);
EXCEPTION
 WHEN NO_DATA_FOUND THEN
  DBMS_OUTPUT.PUT_LINE('Employee not found');
 WHEN TOO_MANY_ROWS THEN
  DBMS_OUTPUT.PUT_LINE('Employee number is not unique');
END;
/
```

`%TYPE` anchors the variable type to the Salary column. SELECT INTO expects one row, so the primary key should identify exactly one employee.

Q10. Explain stored procedures and their advantages

A stored procedure is a named PL/SQL program unit stored and compiled in the database. It can accept IN, OUT, and IN OUT parameters and perform operations such as updates, validations, and logging.

Advantages:

- **Reuse:** one implementation can be called from multiple applications.
- **Performance:** fewer network round trips; compiled program units can reuse parsed code where applicable.
- **Security:** users can be granted EXECUTE privilege without direct access to underlying tables, when designed with appropriate rights.
- **Maintainability:** business rules are centralized.

- **Modularity:** large logic is divided into smaller units.

Example skeleton:

```
CREATE OR REPLACE PROCEDURE greet_user(p_name IN VARCHAR2) AS
BEGIN
 DBMS_OUTPUT.PUT_LINE('Hello ' || p_name);
END;
/
```

Q11. Create a stored procedure to increase an employee's salary by a specified percentage

```
CREATE OR REPLACE PROCEDURE increase_salary (
 p_empno IN Employee.EmpNo%TYPE,
 p_percent IN NUMBER
) AS
BEGIN
 IF p_percent < 0 THEN
  RAISE_APPLICATION_ERROR(-20001, 'Percentage cannot be negative');
 END IF;

 UPDATE Employee
 SET Salary = Salary * (1 + p_percent / 100)
 WHERE EmpNo = p_empno;

 IF SQL%ROWCOUNT = 0 THEN
  RAISE_APPLICATION_ERROR(-20002, 'Employee not found');
 END IF;
END;
/
```

Call it with `EXEC increase_salary(100, 5);` in SQL*Plus/SQL Developer, or through an anonymous PL/SQL block. The caller should decide whether and when to COMMIT or ROLLBACK, according to the application's transaction policy.

Q12. Difference between stored procedures and functions

Procedure: designed mainly to perform an action; it may have IN, OUT, and IN OUT parameters and does not have to return a value through `RETURN`. Function: designed to compute and return a value using a RETURN type and a RETURN statement.

- **Return value:** a function must return a value of its declared type; a procedure does not need to return a function value.
- **Invocation:** a function can often be used in expressions and SQL contexts subject to SQL-callable-function rules; a procedure is invoked as a statement or PL/SQL call.
- **Parameters:** both may use parameter modes supported by PL/SQL, but SQL invocation of functions imposes restrictions.
- **Typical purpose:** procedure for a business action such as updating salary; function for a calculation such as annual salary.

Neither is automatically ‘faster’; choose based on purpose, interface, transaction behavior, and reuse.

Q13. Explain PL/SQL functions and their structure with an example

A function is a named PL/SQL program unit that returns one value through a RETURN clause and a RETURN statement. It can accept parameters and contain declarations, executable code, and exception handlers.

Structure:

```
CREATE OR REPLACE FUNCTION annual_salary (
 p_empno IN Employee.EmpNo%TYPE
) RETURN NUMBER AS
 v_salary Employee.Salary%TYPE;
BEGIN
 SELECT Salary INTO v_salary
 FROM Employee
 WHERE EmpNo = p_empno;
 RETURN v_salary * 12;
EXCEPTION
 WHEN NO_DATA_FOUND THEN
  RETURN NULL;
END;
/
```

Call from PL/SQL: `v_yearly := annual_salary(100);`. It may be called from SQL if it satisfies the DBMS rules for SQL-callable functions. Decide deliberately how missing employees and NULL salary values should be handled.

Q14. Compare procedures and functions based on purpose, parameters, and return values

Purpose: procedure executes an operation (update, insert, workflow); function primarily computes and returns a result.

Parameters: both support parameter passing in PL/SQL. IN passes a value in; OUT returns a value; IN OUT passes a value both ways. However, when a function is invoked from a SQL statement, only SQL-compatible parameter/return behavior is permitted and side effects are restricted.

Return value: a function declares `RETURN datatype` and must execute a compatible RETURN statement; a procedure has no mandatory return value, though it can supply outputs with OUT/IN OUT parameters.

Example: `increase_salary(empno, percentage)` is naturally a procedure; `annual_salary(empno) RETURN NUMBER` is naturally a function.

Q15. Is a procedure or function more suitable for calculating and returning annual salary? Justify

A function is usually more suitable because the primary goal is to calculate and return a value. For example, `annual_salary(p_empno) RETURN NUMBER` can return monthly salary × 12 and be called from PL/SQL expressions or, subject to Oracle SQL-callable-function restrictions, a SQL query.

A procedure is more appropriate when the task primarily performs an action, such as changing an employee's salary or producing several OUT values. Use a function when a single computed result is the main purpose. Handle missing employee rows and NULL salaries explicitly, and avoid hidden side effects in functions intended for SQL use.

Q16. Explain cursors and differentiate implicit and explicit cursors

A cursor is a handle to the context area used to process a SQL statement/result set.

Implicit cursor: Oracle creates it automatically for DML statements and single-row SELECT INTO. Attributes include SQL%ROWCOUNT, SQL%FOUND, SQL%NOTFOUND, and SQL%ISOPEN. Example: after UPDATE, `IF SQL%ROWCOUNT = 0 THEN ...`.

Explicit cursor: declared by the programmer for a query that may return multiple rows. Typical steps: DECLARE cursor, OPEN, FETCH rows, EXIT when `%NOTFOUND`, CLOSE.

```
DECLARE
 CURSOR c_emp IS SELECT EmpNo, Name FROM Employee;
 v_empno Employee.EmpNo%TYPE;
 v_name   Employee.Name%TYPE;
BEGIN
 OPEN c_emp;
 LOOP
  FETCH c_emp INTO v_empno, v_name;
  EXIT WHEN c_emp%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE(v_empno || ': ' || v_name);
 END LOOP;
 CLOSE c_emp;
END;
/
```

A cursor FOR loop can manage open/fetch/close automatically. Explicit cursors offer control over multirow processing; implicit cursors are convenient for individual statements.

Q17. Explain database triggers and their different types with examples

A trigger is a stored program that runs automatically when a defined database event occurs. In Oracle, common DML triggers respond to INSERT, UPDATE, or DELETE.

- **BEFORE trigger:** runs before the triggering change; useful for validation or setting values.
- **AFTER trigger:** runs after the change succeeds; useful for audit logging.
- **Row-level trigger:** includes `FOR EACH ROW` and executes once for each affected row; can reference `:NEW` and `:OLD`.
- **Statement-level trigger:** executes once per SQL statement, even if it affects many rows; does not use per-row values in the same way.
- **INSTEAD OF trigger:** commonly used on views to implement DML that is not directly updateable.
- **DDL/database event triggers:** can respond to events such as CREATE/ALTER or logon, where supported.

Example audit trigger concept: AFTER UPDATE ON Employee FOR EACH ROW inserts old and new salary values into an audit table. Use triggers carefully because they add implicit behavior and may affect performance.

Q18. Create a trigger to prevent an employee's salary from being updated to a negative value

```
CREATE OR REPLACE TRIGGER trg_salary_nonnegative
BEFORE INSERT OR UPDATE OF Salary ON Employee
FOR EACH ROW
BEGIN
 IF :NEW.Salary < 0 THEN
  RAISE_APPLICATION_ERROR(-20010, 'Salary cannot be negative');
 END IF;
END;
/
```

The trigger fires before INSERT or an update to Salary and rejects the operation if the new value is negative. If NULL salary is also invalid, add `:NEW.Salary IS NULL`. For a simple nonnegative rule, a `CHECK (Salary >= 0)` constraint is usually preferable; the trigger illustrates procedural validation as asked.

Q19. Differentiate BEFORE and AFTER triggers with examples

BEFORE trigger runs before the triggering DML is applied to the row. It is useful for validating or modifying `:NEW` values before storage. Example: reject a negative salary or automatically normalize a value.

AFTER trigger runs after the row change has passed relevant checks and the DML operation has occurred, within the same transaction. It is useful for recording an audit event or performing a related action. Example: after salary changes, insert the old and new salary into an audit table.

Both may be row-level or statement-level in Oracle. An AFTER trigger does not mean the transaction has been committed; a later rollback generally rolls back the triggering DML and transactional effects performed by the trigger.

MODULE 6: TRANSACTION MANAGEMENT AND QUERY PROCESSING

ACID, schedules, locks, recoverability, query execution, and deadlocks.

Q1. Evaluate the importance of transaction states for consistency and reliability

Transaction states describe progress from start to completion or failure. They help the DBMS decide what resources to hold, when to commit changes, and whether recovery or rollback is required.

Importance:

- Active identifies work currently executing.
- Partially committed means the final statement has executed but commit durability may not yet be complete.
- Committed means the transaction completed successfully and its effects are durable.
- Failed identifies a transaction that cannot continue.
- Aborted means its effects have been rolled back; it may be restarted or terminated.

Tracking these states supports atomicity, recovery after crashes, lock/resource release, and correct reporting to applications. A transaction must not be reported as committed before its commit is safely established.

Q2. Explain different states of a database transaction with a neat diagram

- **Active:** transaction is executing reads/writes.
- **Partially committed:** the last statement executed; system is completing commit processing.
- **Committed:** changes are successfully committed and survive failures covered by the DBMS guarantees.
- **Failed:** normal execution cannot proceed due to an error, deadlock victim selection, or system failure.
- **Aborted:** rollback has undone the transaction's effects. It may restart or terminate.

Diagram:

```
        last statement
Active ------------------> Partially committed
  |                          |
  | error/failure            | commit succeeds
  v                          v
Failed                     Committed
  |
  | rollback/undo
  v
Aborted ---- restart ----> Active
   \---- terminate
```

Some textbooks also show a terminated state after commit/abort; the core concepts remain the same.

Q3. Explain ACID properties with examples

ACID stands for Atomicity, Consistency, Isolation, and Durability.

- **Atomicity:** all operations in a transaction happen, or none do. Example: a bank transfer debits one account and credits another; if the credit fails, the debit is rolled back.
- **Consistency:** every committed transaction preserves database constraints and valid business rules. Example: account balance cannot become negative if the schema/business rule prohibits it.

- **Isolation:** concurrent transactions behave as if appropriately isolated from one another; intermediate changes should not cause incorrect results. Example: one user should not read a transfer's half-completed state under an isolation level that forbids dirty reads.
- **Durability:** after COMMIT, changes survive a crash according to the DBMS's durability guarantees. Example: a committed payment remains recorded after restart.

Together ACID makes transaction processing reliable despite errors and concurrency.

Q4. Evaluate the importance of each ACID property

Atomicity prevents partial operations, especially in multi-step work such as transfers or order creation. Without it, a failure could leave only the debit or only the shipment record.

Consistency ensures that transactions preserve constraints, references, and defined business rules. It is supported by keys, checks, triggers when needed, and correct transaction logic.

Isolation prevents harmful interaction between concurrent transactions. It limits anomalies such as dirty reads, lost updates, and inconsistent analyses, according to the isolation level. Stronger isolation may reduce concurrency.

Durability ensures committed results are not lost after a power, process, or system failure, using logging, checkpoints, replication, and storage guarantees.

No single property substitutes for the others: atomicity handles all-or-nothing execution, consistency protects valid states, isolation handles concurrent interaction, and durability protects committed results.

Q5. Explain concurrent execution of transactions and its advantages

Concurrent execution interleaves operations from multiple transactions instead of completing one whole transaction before starting the next. The DBMS scheduler coordinates operations using locks, timestamps, validation, or other concurrency-control methods.

Advantages:

- **Higher throughput:** more transactions finish per unit time.
- **Better resource utilization:** CPU can work while another transaction waits for disk/network I/O.
- **Lower average response time:** users need not wait for a long transaction to finish before short tasks start.
- Supports many simultaneous users.

Correctness requires isolation/concurrency control. Uncontrolled interleaving can cause lost updates, dirty reads, non-repeatable reads, and inconsistent summaries. A serializable schedule is equivalent to some serial ordering.

Q6. Analyze a concurrent schedule for lost updates and dirty reads

Lost update example: initial X=100. T1 adds 10, T2 adds 20.

```
T1: R(X=100)
T2:         R(X=100)
T1: W(X=110)
T2:         W(X=120)
```

The final value is 120, so T1's update is lost. A correct serial execution would yield 130. Locking or an atomic update can prevent it.

Dirty read example: T1 writes X=150 but has not committed. T2 reads X=150. T1 then aborts and restores X=100. T2 has used data that never committed.

```
T1: W(X=150) -------- ABORT; restore X=100
T2:      R(X=150)   (dirty read)
```

Prevention: lock-based protocols and appropriate isolation levels prevent these anomalies. Exact phenomena permitted depend on the DBMS and isolation mode.

Q7. Advantages and disadvantages of concurrent execution compared with serial execution

Serial execution runs transactions one after another and is easy to reason about, but may leave CPU or I/O resources idle while one transaction waits. It generally has lower concurrency and throughput.

Concurrent execution improves throughput, resource use, responsiveness, and support for multiple users. However, it introduces scheduling overhead and possible anomalies if concurrency is not controlled. Locking can cause blocking or deadlocks; strict isolation may reduce performance; recovery must account for interleaved updates.

A good DBMS allows concurrency while guaranteeing a defined isolation/correctness level. Serializability provides a useful correctness criterion for many schedules.

Q8. Explain conflict serializability and when two operations conflict

Two operations conflict if they belong to different transactions, access the same data item, and at least one operation is a write. Therefore RW, WR, and WW pairs conflict; RR does not.

A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping adjacent non-conflicting operations. Use a precedence/serialization graph:

1. Make a node for each transaction.
2. Add edge Ti -> Tj when an operation of Ti conflicts with and occurs before an operation of Tj on the same item.
3. If the graph has no directed cycle, the schedule is conflict-serializable. A topological ordering gives an equivalent serial order. A cycle means it is not conflict-serializable.

Example: R1(X), W1(X), R2(X), W2(X) gives edge T1 -> T2, so it is equivalent to serial order T1 then T2.

Q9. Explain view serializability and conditions for view-equivalent schedules

Two schedules are view-equivalent if, for every data item X:

1. The same transaction reads the initial value of X in both schedules, if any transaction reads the initial value.
2. Each read of X by transaction T reads the value written by the same transaction in both schedules (the same reads-from relationship).
3. The same transaction performs the final write of X in both schedules.

A schedule is view-serializable if it is view-equivalent to a serial schedule. View serializability is more general than conflict serializability; some schedules with blind writes are view-serializable but not conflict-serializable. Testing view serializability is more computationally complex.

Conflict-serializable schedules are always view-serializable, but the reverse is not always true.

Q10. Explain shared locks, exclusive locks, and two-phase locking (2PL)

Shared lock (S): required to read a data item. Multiple transactions may hold S locks on the same item simultaneously, provided no incompatible lock is held.

Exclusive lock (X): required to write a data item. Only one transaction can hold an X lock on an item; it is incompatible with both S and X locks held by other transactions.

Compatibility: S with S = compatible; S with X = incompatible; X with X = incompatible.

Two-phase locking (2PL) has two phases:

- **Growing phase:** transaction may acquire locks but cannot release any.
- **Shrinking phase:** transaction may release locks but cannot acquire new ones.

2PL ensures conflict serializability. Strict 2PL holds exclusive locks (often all write locks) until commit/abort, helping prevent cascading aborts and improving recoverability. Basic 2PL can still lead to deadlocks. Example: T1 locks X, T2 locks Y; T1 waits for Y while T2 waits for X.

Q11. Explain shadow paging with a neat diagram

Shadow paging is a recovery technique that maintains a stable shadow page table and a current page table. At transaction start, the current table initially points to the same database pages as the shadow table. When a page is changed, the system writes the new contents to a different page and updates only the current page table. The shadow table continues pointing to the old stable pages.

```
Before transaction:        During updates:
Shadow table ----> pages   Shadow table ----> old pages
Current table --> same     Current table --> modified/new pages


COMMIT: atomically switch the database root pointer
to the new current page table.
ABORT/CRASH before commit: discard current table/new pages
and use the unchanged shadow table.
```

Advantages: can recover by discarding uncommitted pages; conceptually simple and does not require undo/redo log records for the page changes themselves. Disadvantages: page copying/fragmentation, garbage collection, concurrency complications, and the need for an atomic root-pointer update. Many production DBMSs use logging-based methods rather than pure shadow paging.

Q12. How does shadow paging restore a consistent state after a transaction fails before committing?

At transaction start, the DBMS preserves the shadow page table and its references to the committed database pages. Changes are written to copy-on-write pages, and the current page table is updated to point to those new pages. The shadow table remains unchanged.

If the transaction fails before commit, the DBMS discards the new/modified pages and current page table, then resumes using the shadow table. Because the shadow table still points to the last committed pages, uncommitted changes are invisible and the previous consistent database state is restored.

If commit succeeds, the root pointer is atomically switched to the current page table. The exact atomic-pointer mechanism and page reclamation strategy are implementation details.

Q13. Explain the major steps involved in query processing in a DBMS

1. Parsing and validation: check SQL syntax, object names, types, and user permissions.
2. Translation: convert the SQL statement into an internal representation such as a relational algebra expression or query tree.
3. Query optimization: compare alternative plans, estimate costs, and choose an efficient access/join order using statistics and indexes.
4. Plan/code generation: produce a physical execution plan with operators and access methods.
5. Query evaluation/execution: run scans, joins, sorts, filters, and aggregates through the execution engine.
6. Result delivery: return rows or report the number of changed rows; transaction and consistency rules are applied throughout.

Optimization matters because equivalent SQL expressions can have very different costs depending on indexes, cardinality, join order, and data distribution.

Q14. Analyze parsing, translation, optimization, and query evaluation

Parsing validates SQL syntax and builds a parse tree. Semantic checking resolves table/column names, data types, functions, and privileges. Errors are returned before execution when possible.

Translation turns the validated query into a logical relational-algebra expression or logical plan. Logical rewrite rules may simplify predicates, push selections/projections down, and transform equivalent expressions.

Optimization considers different equivalent physical plans. It estimates disk I/O, CPU, memory, row counts, available indexes, join algorithms, and statistics. It chooses an expected low-cost plan; estimates are not perfect, so stale statistics can lead to poor plans.

Evaluation is the actual execution of the chosen plan: scan pages/indexes, apply predicates, join relations, aggregate, sort, and produce the result. Transaction management, buffers, and concurrency control coordinate access while it runs.

Q15. Analyze the necessary conditions for deadlock in a DBMS

A deadlock occurs when transactions wait indefinitely for locks or resources held by one another. Four Coffman conditions must all hold for a deadlock to be possible:

- **Mutual exclusion:** at least one resource/lock is non-shareable.
- **Hold and wait:** a transaction holds one resource while waiting for another.
- **No preemption:** a resource cannot simply be taken away; it is released by its holder or through abort/rollback.
- **Circular wait:** a cycle exists, such as T1 waits for T2, T2 waits for T3, and T3 waits for T1.

Example: T1 holds lock A and waits for B; T2 holds B and waits for A.

Prevention breaks one of the conditions; avoidance uses knowledge of resource requirements; detection builds a wait-for graph and looks for cycles; recovery selects a victim transaction to abort. Timeouts are a practical but imperfect detection approach.

Q16. Explain shared locks, exclusive locks, and two-phase locking (duplicate question)

Shared (S) locks allow concurrent readers and are compatible with other S locks. Exclusive (X) locks are needed for writing and conflict with both S and X locks held by another transaction.

2PL separates locking into a growing phase (acquire locks; release none) and a shrinking phase (release locks; acquire no new ones). It guarantees conflict serializability. Strict 2PL keeps write locks until commit/abort, reducing cascading aborts and making recovery safer.

Example: T1 obtains X-lock(A), updates A, then holds the lock until commit. T2 requesting S-lock(A) must wait. Once T1 commits and releases the lock, T2 can read the committed value. 2PL does not eliminate deadlocks, so a DBMS must detect, prevent, or resolve them.

Last-minute revision sheet

- **Keys:** Super key uniquely identifies; candidate key is minimal superkey; primary key is chosen candidate key; foreign key references a parent key; UNIQUE rejects duplicate values.
- **ER cardinality:** 1:1 = one on each side; 1:N = one parent to many children; M:N = many on each side and normally requires a junction table. State total/partial participation too.
- **Relational algebra:** σ = select rows; π = project columns; ρ = rename; × = Cartesian product; ⋈ = join; ∪ = union; ∩ = intersection; − = difference.
- **SQL clauses:** WHERE filters rows; GROUP BY forms groups; HAVING filters groups; ORDER BY sorts final output.
- **SQL commands:** DDL: CREATE/ALTER/DROP/TRUNCATE. DML: INSERT/UPDATE/DELETE (SELECT is often called DQL). DCL: GRANT/REVOKE. TCL: COMMIT/ROLLBACK/SAVEPOINT.
- **Normal forms:** 1NF: atomic values. 2NF: no partial dependency of non-prime attributes on a candidate key. 3NF: for every nontrivial X->A, X is a superkey or A is prime. BCNF: every determinant of a nontrivial FD is a superkey.
- **Closure and keys:** To compute X+, start with X and repeatedly add attributes implied by FDs. If X+ contains every attribute, X is a superkey; if minimal, it is a candidate key.
- **PL/SQL block:** DECLARE (optional) -> BEGIN (required) -> EXCEPTION (optional) -> END; / . SELECT INTO must return exactly one row.
- **Cursor:** Implicit cursor is managed by Oracle for a statement; explicit cursor is declared by the programmer for controlled multirow processing.
- **Trigger:** BEFORE can validate/change :NEW before DML; AFTER can audit after the row change; AFTER does not mean transaction committed.
- **Transaction states:** Active -> Partially committed -> Committed. Failure -> Failed -> Aborted; the transaction may restart.
- **ACID:** Atomicity all or nothing; Consistency valid state to valid state; Isolation controls concurrency effects; Durability committed data survives covered failures.
- **Conflict serializability:** Conflicts are cross-transaction operations on the same item with at least one write (RW, WR, WW). Precedence graph acyclic => conflict-serializable.
- **Deadlock:** All four: mutual exclusion, hold-and-wait, no preemption, circular wait.
- **Query processing:** Parse/validate -> translate/rewrite -> optimize -> generate plan -> evaluate/execute -> return results.